LLDD BE - API Lookup

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจทาน และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน
และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. Overview

รายการ	รายละเอียด
Track	BE
Estimate	13 ชั่วโมง = implementation 10 + unit test 3 (30%)
Owner	Tunyatorn <Vava> Kiatkongphongsa
Target repository	`SBP/srm-sps-spsap-store-backend` (NestJS + TypeORM · schema `sps_store`) + `SBP/srm-sps-spsap-sbp-bff` (forward ผ่าน client service · ไม่มี DB) สำหรับเส้นที่ FE เรียก
Objective	ออกแบบ APIs กลุ่ม lookup ที่ใช้ร่วมทุกหน้าจอของ SBP Mall

Common contract reference: ทุกหัวข้อ API/FE ต้องยึด LLDD-BE-API-Common-Contracts.pdf และ LLDD-FE-Integration-Contracts.pdf สำหรับ error/auth/format/pagination/action/RBAC ก่อนลงรายละเอียดเฉพาะหน้าหรือเฉพาะ endpoint

1.1 เอกสาร LLDD ที่เกี่ยวข้อง

ตารางนี้สร้างจาก endpoint และตารางที่เอกสารฉบับนี้ประกาศไว้จริง — อ่านฉบับที่อยู่ในตารางก่อนลงมือ เพื่อไม่ให้สัญญา request/response หรือชื่อคอลัมน์หลุดจากกัน

ความสัมพันธ์	เอกสาร LLDD	เกี่ยวข้องตรงไหน
ถูกเรียกจาก	LLDD-FE-Create-Document.pdf	GET /store/search (ระบบ SBP เดิม)
ถูกเรียกจาก	LLDD-FE-Foundation.pdf	`GET /api/v1/sgi/lookup/document-statuses`
ถูกเรียกจาก	LLDD-FE-Report.pdf	GET /store/search (ระบบ SBP เดิม)
สัญญากลาง	LLDD-BE-API-Common-Contracts.pdf	envelope `{success, data}` · error code · pagination · รูปแบบวันที่/เลขเอกสาร
โครงสร้างข้อมูล	LLDD-BE-Database-Structure.pdf	DDL ของตารางที่หัวข้อ Reference DB Mapping อ้างถึง
แพลตฟอร์มระบบเดิม	LLDD-BE-Integration-SBP-Platform.pdf	header จาก BFF (`x-api-key` / `x-user-*`) · การ reuse ตารางและ service ของระบบ SBP เดิม
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-API-Common-Contracts.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-Database-Structure.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง

2. Screen / Functional Scope

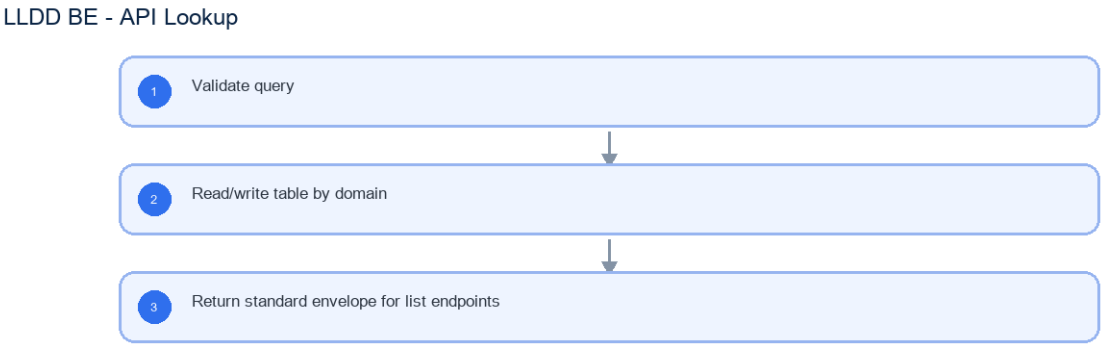
- Lookup APIs
- Auth endpoints are platform reference only

3. Screenshot Reference

ไม่มีภาพหน้าจอสำหรับหัวข้อนี้ — เป็นเอกสารฝั่ง Backend/Batch ที่ไม่มี UI (ภาพหน้าจอทั้งหมดอยู่ในเอกสารชุด FE)

4. Implementation Flow & Sequence Diagram (Reference)

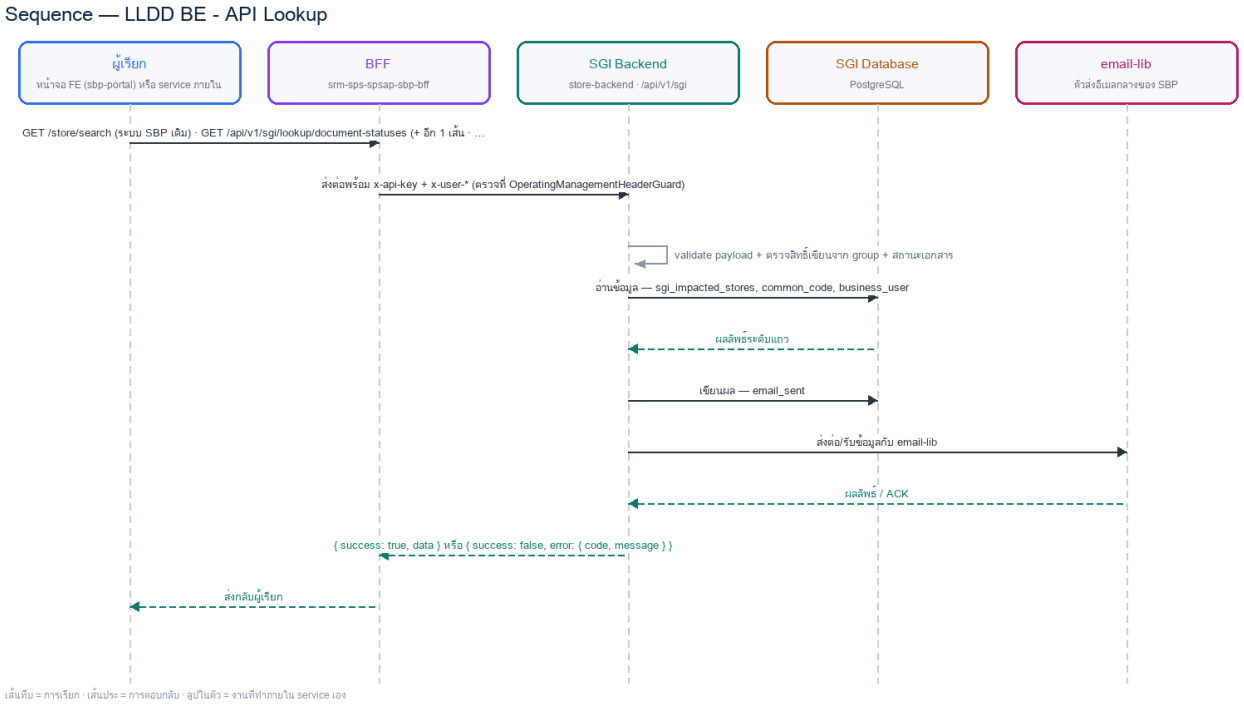
4.1 Implementation Flow (ลำดับขั้นการทำงาน)



รูปที่ 1: Implementation flow reference: LLDD BE - API Lookup

4.2 Sequence Diagram (ใครคุยกับใคร ลำดับไหน)

ผู้แสดงและลำดับข้อความในภาพนี้สร้างจาก endpoint ในหัวข้อ 7 และตารางในหัวข้อ Reference DB Mapping ของเอกสารฉบับนี้เอง จึงตรงกับสัญญาเสมอ



รูปที่ 2: Sequence diagram: LLDD BE - API Lookup

5. Field, Format, and Validation

Field / UI	Format	Validation	Behavior
q	string	optional	ใช้ค้นหา ร้าน (store/mas_store ระบบเดิม) · พนักงาน (business_user ระบบเดิม) · คู่แข่ง (sgi_competitors ของ SGI)
type	impacted|new	required for /store/search (ระบบ SBP เดิม)	เลือกแหล่งร้านถูกกระทบ/ร้านเปิดใหม่
roleCode	00-10	required for permission	group ของ auth-backend (ระบบเดิม) — SGI ไม่มีตาราง roles

Field / UI	Format	Validation	Behavior
menuCode	string	required for permission	menu ของ auth-backend (ระบบเดิม) — SGI ไม่มีตาราง menus
templateCode	EM-01..EM-08	required	email template key
reason	text	ไม่บังคับแล้ว	ไม่มีปลายทางเก็บ (ยกเลิกระบบ audit ของ master 2026-08-07)

5.9 Input / Progress / Output Contract

Stage	Contract for implementation
Input	GET /store/search (ระบบ SBP เดิม); GET /api/v1/sgi/lookup/document-statuses; GET /api/v1/sgi/lookup/workflow-sections
Progress	Validate query; Read/write table by domain; Return standard envelope for list endpoints
Output	ไม่มีตารางที่เอกสารนี้เขียนเอง — output คือ response ตาม envelope กลาง `{success, data}` และร่องรอยที่ตรวจย้อนได้ (log / sgi_consideration_logs / workflow_history ของ engine)

5.90 Endpoint Implementation Contract

Endpoint	Use-case owner	Service/repository behavior	Definition of done
GET /store/search (ระบบ SBP เดิม)	ค้นหาร้านสำหรับ popup	Validate query	status label ต้องเป็น verbatim
GET /api/v1/sgi/lookup/document-statuses	รายการสถานะเอกสาร verbatim	Read/write table by domain	permission mutation ต้อง audit
GET /api/v1/sgi/lookup/workflow-sections	รายการ section 5 ขั้น	Return standard envelope for list endpoints	SGI เรียก email-lib ส่งอีเมลเอง โดยใช้เลข template จาก workflow_route.email_id (ปิด DP-5 · 2026-08-14)

5.91 Backend Execution Sequence

Step	Behavior specific to this LLDD	Failure/test evidence
1	Validate query	store lookup
2	Read/write table by domain	status lookup
3	Return standard envelope for list endpoints	permission save without reason

6. Button / User Action Mapping

Action	Trigger	API / Service	Expected Result
Store lookup	GET	lookup.service.searchStores	return impacted/new stores
Employee lookup	GET	employee backend เดิมของระบบ SBP (ไม่ใช่ endpoint ของ SGI)	return business_user for operator popup
Permission save	PUT	rbac.service.saveMenuPermission	update can_access and audit
Email template save/reset	PUT/POST	notificationTemplate.service	update/reset template and audit

7. API Contract

GET /store/search (ระบบ SBP เดิม)

ค้นหาร้านสำหรับ popup

Query Params

```
{
  "q": "00788",
  "type": "impacted",
  "activeOnly": false
}
```

Request Field Schema

Field	Type	Required	Constraint / Meaning
q	string	No	ช่อง ค้นหา ของหน้า master — ค้นแบบ contains ไม่สนตัวพิมพ์ ในรหัส + ชื่อ + รายละเอียด (คู่แข่งเพิ่มชื่ออังกฤษด้วย) · ● ต้องครอบรหัสด้วยเสมอ เพราะช่องบนหน้าจอเขียนว่า "ค้นหารหัส / ชื่อ…" · หน้าจอส่งค่านี้หลังหน่วง 350ms และ รีเซ็ต page กลับ 1 ทุกครั้งที่คำค้นเปลี่ยน
type	string	No	UTF-8; use value domain described by endpoint purpose
activeOnly	boolean	No	● หน้าจอ SGI ต้องส่ง `activeOnly=false` (เพิ่ม 2026-09-24) — ไม่ส่ง = `true` ตามพฤติกรรมเดิมของหน้าจออื่น · BFF กรองเฉพาะร้านที่ยังเปิดดำเนินการ แต่ ร้านที่มีเอกสารประกันรายได้แทบทั้งหมดปิดไปแล้ว (วัดบน dev: ร้านที่มีเอกสาร SGI ไม่มีสักร้านที่ `active_flag = Y`) ค้นรหัสร้านที่มีข้อมูลจริงจึงขึ้น "ไม่พบข้อมูล" ทุกครั้ง (ผู้ใช้แจ้งด้วยร้าน 00256)

Response

```
{
  "items": [
    {
      "storeCode": "00788",
      "storeName": "รัตนอุทิศ ซ.13"
    }
  ]
}
```

Response Field Schema

Field	Type	Required	Constraint / Meaning
items	array<object>	Yes	JSON array; element type shown in Type column
items[].storeCode	string	Yes	exactly 5 digits; preserve leading zero
items[].storeName	string	Yes	UTF-8; use value domain described by endpoint purpose

GET /api/v1/sgi/lookup/document-statuses

รายการสถานะเอกสาร verbatim

Query Params

```
{}
```

Request Field Schema

Field	Type	Required	Constraint / Meaning
-	none	No	No fields

Response

```
{
  "items": [
    {
      "statusCode": "06",
      "currentOwner": "somchai.k",
      "statusName": "รอฝ่าย SBP DSA ดำเนินการ",
      "code": "06",
      "label": "รอฝ่าย SBP DSA ดำเนินการ"
    }
  ]
}
```

Response Field Schema

Field	Type	Required	Constraint / Meaning
items	array<object>	Yes	JSON array; element type shown in Type column
items[].statusCode	string	Yes	canonical code; do not replace with display label
items[].currentOwner	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].statusName	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].code	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].label	string	Yes	UTF-8; use value domain described by endpoint purpose

GET /api/v1/sgi/lookup/workflow-sections

รายการ section 5 ขั้น

Query Params

```
{}
```

Request Field Schema

Field	Type	Required	Constraint / Meaning
-	none	No	No fields

Response


```
{
  "items": [
    {
      "sectionCode": "06",
      "sectionName": "ฝ่าย SBP DSA"
    }
  ]
}
```


Response Field Schema

Field	Type	Required	Constraint / Meaning
items	array<object>	Yes	JSON array; element type shown in Type column
items[].sectionCode	string	Yes	canonical code; do not replace with display label
items[].sectionName	string	Yes	UTF-8; use value domain described by endpoint purpose

8. Reference DB Mapping (No Database Page Work)

ส่วนนี้เป็นข้อมูลอ้างอิงสำหรับการ implement API/Job เท่านั้น ไม่ใช่งานสร้างหน้า Database, ไม่ใช่งานออกแบบ DB page และไม่ถูกนับเป็น deliverable แยกของ FE/BE

Table / Object	R/W	Usage
sgi_impacted_stores (SGI) / store · mas_store · sevenshop (SBP เดิม)	R	store picker — SGI ไม่มีตาราง stores ของตัวเอง
common_code (SBP เดิม · SGI_DOC_STATUS / SGI_APPROVE_LIMIT)	R	● แก้ 2026-09-16 — lookup สถานะ 6 ค่าและขั้น 5 ค่าอ่านจาก common_code ไม่ใช่ sps_store.workflow_status เพราะตารางนั้นไม่มีคอลัมน์ลำดับ และ status_id เป็น integer surrogate ต่างกันตาม environment · วงเงินอนุมัติอยู่ที่ code_name ของ SGI_APPROVE_LIMIT
workflow_status / workflow_state (@srm/glb-workflow · sps_store)	R	ชื่อสถานะฝั่ง engine — engine เป็นเจ้าของสถานะตอนรันไทม์ · lookup API ไม่ได้อ่านตารางนี้แล้ว
business_user (SBP เดิม)	R	popup ค้นหาพนักงาน — SGI ไม่มีตาราง employees
auth-backend groups / menus / permissions (ระบบเดิม)	R	RBAC/menu matrix — จัดการที่หน้า /setting/manage-user-rights เดิม · SGI อ่านผ่าน header x-user-permissions เท่านั้น ไม่มีตารางของตัวเอง
email_template (SBP เดิม)	R	template — SGI อ่านผ่าน lib เท่านั้น ไม่แก้ของระบบเดิม
email_sent (SBP เดิม)	W (โดย email-lib)	log การส่ง — SGI เรียก sendEmail() ด้วยเลข template จาก workflow_route.email_id แล้ว lib เขียนแถวให้เอง (DP-5 · 2026-08-14) · ▲ คอลัมน์ผู้ส่งคือ send_by ไม่ใช่ sent_by

9. Skeleton Code (store-backend + BFF)

โครงโค้ดตั้งต้นของเอกสารฉบับนี้ ยึด convention จริงของ `srm-sps-spsap-store-backend` (NestJS 11 + TypeORM, schema `sps_store`, custom provider `DATA_SOURCE` ที่ route SELECT ไป slave pool) และ `srm-sps-spsap-sbp-bff` (ไม่มี DB, forward ผ่าน client service). ทุกจุดที่ต้องเติมกำกับด้วย `// TODO:` และ response ทุกเส้นถูกห่อเป็น `{success, data}` โดย ResponseInterceptor อยู่แล้ว จึงห้าม service ห่อซ้ำ

9.1 ผังไฟล์ที่ต้องสร้าง

Path	หน้าที่
store-backend · src/modules/sgi-lookup/sgi-lookup.controller.ts	route ทั้งหมดของเอกสารนี้ (2 เส้น) + `@UseGuards(HttpHeaderGuard)` + `@UserId()`
store-backend · src/modules/sgi-lookup/sgi-lookup.service.ts	business logic — inject `'DATA_SOURCE'` แล้วยิง raw SQL, mutation ใช้ QueryRunner transaction
store-backend · src/modules/sgi-lookup/sgi-lookup.sql.ts	เก็บ SQL ต่อ endpoint (คัดจากหัวข้อ 10) แยกออกจาก service ให้ทดสอบ/รีวิวง่าย · คีย์ = ชื่อ handler เช่น `getSgiMasterFactors` · บล็อกที่มีหลาย statement ให้แยกเป็นหลายคีย์ โดยเติมท้ายชื่อให้สื่อความ เช่น DELETE master ที่มี 2 statement → `removeSgiMasterFactorsByCodeInUse` (SELECT ตรวจการใช้งาน) + `removeSgiMasterFactorsByCode` (DELETE)
store-backend · src/modules/sgi-lookup/dto/sgi-lookup.dto.ts	DTO + class-validator ตาม validation ในหัวข้อฟิลด์ของเอกสารนี้
store-backend · src/modules/sgi-lookup/sgi-lookup.module.ts	ประกอบ controller/service/providers แล้ว register ที่ `app.module.ts`
store-backend · src/entitys/sgi-impacted-stores.entity.ts	entity ของ `sgi_impacted_stores` (`@Entity({schema: process.env.DB_SCHEMA})`, ไม่ประกาศ relation)
store-backend · src/entitys/common-code.entity.ts	entity ของ `common_code` (`@Entity({schema: process.env.DB_SCHEMA})`, ไม่ประกาศ relation)
store-backend · src/entitys/business-user.entity.ts	entity ของ `business_user` (`@Entity({schema: process.env.DB_SCHEMA})`, ไม่ประกาศ relation)
store-backend · src/providers/sgi/sgi.ts	repository provider แบบ factory ผูก token string กับ `DATA_SOURCE` — ไฟล์ร่วมของทุกเอกสาร BE ให้ merge array เพิ่ม ห้ามเขียนทับ
store-backend · sql/deploy-sgi-lookup.sql	DDL production แบบ idempotent (ทีมนี้ไม่ใช้ migration เป็นหลัก)
BFF · src/common/client-services/sgi-client.service.ts	client ต่อจาก `BaseClientService` ตั้ง baseUrl + `x-api-key` ตอน `onModuleInit`
BFF · src/modules/sgi-lookup/sgi-lookup.controller.ts	route ฝั่ง BFF prefix /bff/sgi/… + `@UseGuards(AuthGuard('jwt'))`
BFF · src/modules/sgi-lookup/sgi-lookup.service.ts	แนบ `x-user-id` / `x-user-group-id` / `x-user-permissions` แล้ว forward ไป backend

เส้นที่ไม่ต้อง implement ใหม่ในเอกสารนี้:

Endpoint	จุดประสงค์	เหตุผล
GET /store/search (ระบบ SBP เดิม)	ค้นหาร้านสำหรับ popup	endpoint ของระบบ SBP เดิม — เรียกใช้ ไม่ต้อง implement ใหม่

9.2 Controller (store-backend)

```
// src/modules/sgi-lookup/sgi-lookup.controller.ts
import { Controller, Get, UseGuards } from '@nestjs/common';
import { HttpHeaderGuard } from '../../guards/http-header.guard';
import { UserId } from '../../common/decorators/user-id.decorator';
import { SgiLookupService } from './sgi-lookup.service';

// LLDD BE - API Lookup
// BFF เรียกด้วย x-api-key และแนบ x-user-id / x-user-group-id / x-user-permissions มาให้
@Controller('lookup')
@UseGuards(HttpHeaderGuard)
export class SgiLookupController {
  constructor(private readonly service: SgiLookupService) {}

  // GET /api/v1/sgi/lookup/document-statuses — รายการสถานะเอกสาร verbatim
  @Get('document-statuses')
```

```
  getSgiLookupDocumentStatuses(@UserId() userId: string) {
    // TODO: ตรวจ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
    return this.service.getSgiLookupDocumentStatuses(userId);
  }

  // GET /api/v1/sgi/lookup/workflow-sections — รายการ section 5 ขั้น
  @Get('workflow-sections')
  getSgiLookupWorkflowSections(@UserId() userId: string) {
    // TODO: ตรวจ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
    return this.service.getSgiLookupWorkflowSections(userId);
  }
}
```

9.3 DTO + Validation

```
// src/modules/sgi-lookup/dto/sgi-lookup.dto.ts
import { Type } from 'class-transformer';
import {
  IsArray, IsBoolean, IsIn, IsInt, IsNotEmpty, IsNumber, IsObject, IsOptional,
  IsString, Matches, Max, MaxLength, Min, ValidateNested,
} from 'class-validator';

// ValidationPipe ระดับ global ตั้ง whitelist + forbidNonWhitelisted + transform ไว้แล้ว (main.ts)
// property ที่ไม่ประกาศที่นี่จะถูก reject เป็น 400 อัตโนมัติ

// payload ของโมดูลนี้
export class LookupRequestDto {
  // TODO: endpoint นี้ไม่มี body/query ใน LLDD — เพิ่ม property เมื่อสรุป payload แล้ว
}
```

9.4 Service (inject `DATA_SOURCE` + raw SQL)

service ประกาศ method ครบทุกเส้นที่ controller เรียก และ signature มาจากแหล่งเดียวกับ controller (จำนวน/ลำดับพารามิเตอร์จึงตรงกันเสมอ) — เส้นที่ยังไม่ได้ implement เป็น stub ที่ `throw new NotImplementedException(...)` ให้ TypeScript compile ผ่านตั้งแต่วันแรก

```
// src/modules/sgi-lookup/sgi-lookup.service.ts
import { Inject, Injectable, Logger, NotImplementedException } from '@nestjs/common';
import { DataSource } from 'typeorm';
import { SgiException } from '../sgi/exceptions/sgi.exception';
import { WorkflowService } from '../workflow/workflow.service';
import { SGI_SQL } from './sgi-lookup.sql';

@Injectable()
export class SgiLookupService {
  private readonly logger = new Logger(SgiLookupService.name);
  // versionId ของ workflow ประกันรายได้ (ตั้งใน env เหมือน COOPERATION_WORKFLOW_VERSION_ID)
  private readonly versionId = Number(process.env.SGI_WORKFLOW_VERSION_ID);

  constructor(
    // DATA_SOURCE override query(): SELECT/WITH ไป slave pool, write ไป master
    @Inject('DATA_SOURCE') private readonly dataSource: DataSource,
    private readonly workflow: WorkflowService,
  ) {}

  // GET /api/v1/sgi/lookup/document-statuses — รายการสถานะเอกสาร verbatim
  async getSgiLookupDocumentStatuses(userId: string) {
    const page = 1;
    // DTO ของเส้นนี้ไม่มี page/size (ดูหัวข้อ DTO) — ไม่แบ่งหน้า
    const size = 100;
    // SQL เต็มอยู่ในหัวข้อ Database SQL ของเอกสารนี้ (คีย์ 'GET /api/v1/sgi/lookup/document-statuses')
    // SQL ในเอกสารเป็น positional $1..$n อยู่แล้ว (ตัวสร้างแปลงให้ตั้งแต่ 2026-09-04)
    // บรรทัดแรกของบล็อก SQL คือ `-- bind ตามลำดับ: $1=... · $2=...` ให้เรียงอาร์กิวเมนต์ตามนั้น
    const rows = await this.dataSource.query(SGI_SQL.getSgiLookupDocumentStatuses, [
      // เรียงให้ตรงกับบรรทัด `-- bind ตามลำดับ:` ของ SQL เส้นนี้
      userId, (page - 1) * size, size,
    ]);
    // TODO: total ต้องมาจาก COUNT(*) แยก query หรือ window function ไม่ใช่ rows.length
    return { page, size, total: rows.length, items: rows };
  }

  // GET /api/v1/sgi/lookup/workflow-sections — รายการ section 5 ขั้น
  async getSgiLookupWorkflowSections(userId: string) {
    // TODO: implement ตาม business rule ของ GET /api/v1/sgi/lookup/workflow-sections
    // (SQL อยู่ในหัวข้อ Database SQL คีย์ 'GET /api/v1/sgi/lookup/workflow-sections')
    throw new NotImplementedException('getSgiLookupWorkflowSections ยังไม่ implement');
  }
}
```

9.5 Workflow (`@srm/glb-workflow`)

✓ ชื่อ function ของ engine — ยึด LLDD ของ lib (ยืนยันแล้ว 2026-08-14) · API จริงคือ 8 ตัวตามชีต `Detail` ของ `SBP/TSM-SRM-LLDD SBP workflow 1.2.xlsx` (เอกสารของ lib เอง): `initializeWorkflow` · `eventWorkflow` · `getPermissionEvents` · `getHistory` · `getTransaction` · `getPendingFlowByUser` · `getWorkflowsByUser` · `addPreApprover` · ชื่อที่เคยขัดกันไม่ใช่ชื่อ API — *Trigger Event* เป็นชื่อหัวข้อขั้นตอนภายใน `eventWorkflow` และ `*UseCase` เป็น class ที่ store-backend ห่อไว้ใช้เอง (ดู `LLDD-BE-Workflow-Engine-Definition.pdf` หัวข้อ 5.3)

Endpoint	Use case ที่ต้องเรียก	เหตุผล
(อ่านสถานะประกอบ)	getTransaction()	อ่านสถานะปัจจุบันของเอกสารเพื่อประกอบ response

```
// src/modules/sgi-lookup/sgi-lookup.workflow.ts (หรือรวมไว้ใน service เดียวกัน)
// WorkflowService = wrapper ของ @srm/glb-workflow ที่ store-backend มีอยู่แล้ว
// (DataSource แยกชื่อ 'workflow-connection', ทุก use case ห่อด้วย TypeOrmUnitOfWork)

 // สถานะปัจจุบันของเอกสาร
 const trx = await this.workflow.getTransaction({ versionId: this.versionId, referenceId: docNo });
 // TODO: map currentState -> statusCode/statusName ที่ FE ใช้
```

9.6 Entity (TypeORM)

```
// src/entitys/sgi-impacted-stores.entity.ts
import { Column, Entity, PrimaryColumn } from 'typeorm';

@Entity({ name: 'sgi_impacted_stores', schema: process.env.DB_SCHEMA })
export class ImpactedStore {
 @PrimaryColumn({ name: 'store_code', type: 'varchar', length: 5 })
 storeCode: string;

 @Column({ name: 'dv_code', type: 'varchar', length: 20, nullable: true })
 dvCode?: string;

 @Column({ name: 'opt_dv_user_id', type: 'varchar', length: 30, nullable: true })
 optDvUserId?: string;

 @Column({ name: 'latitude', type: 'numeric', precision: 10, scale: 7, nullable: true })
 latitude?: string;

 @Column({ name: 'longitude', type: 'numeric', precision: 10, scale: 7, nullable: true })
 longitude?: string;

 @Column({ name: 'transfer_sbp_date', type: 'date', nullable: true })
 transferSbpDate?: Date;

 @Column({ name: 'is_active', type: 'boolean', default: true })
 isActive: boolean;

 @Column({ name: 'updated_at', type: 'timestamp' })
 updatedAt: Date;

 // entity ชุดนี้ generate จาก DDL ใน LLDD-Database.pdf §5.2–5.4 โดยตรง — คอลัมน์/ชนิด/nullable ตรงกันเสมอ
 // ไม่ประกาศ relation ตาม convention ของทีม (join ด้วย raw SQL)
`
// src/entitys/common-code.entity.ts
import { Column, Entity, PrimaryColumn } from 'typeorm';

@Entity({ name: 'common_code', schema: process.env.DB_SCHEMA })
export class CommonCode {
 @PrimaryColumn({ name: 'id', type: 'bigint' })
 id: number;

 // TODO: เติมคอลัมน์ที่เหลือของ common_code ตาม Database design (Canonical Column Contract)
 // และห้ามประกาศ relation — โมดูลนี้ join ด้วย raw SQL ตาม convention ของทีม
}
```

ตารางที่เหลือของเอกสารนี้ (`business_user`, `permissions`, `email_sent`) ใช้รูปแบบ entity เดียวกัน — คอลัมน์อ้างจาก `Database design`

ตารางที่ ไม่ต้องสร้าง entity เพราะใช้ของระบบเดิม/workflow engine:

Object	R/W	ใช้ของระบบเดิมตัวไหน
workflow_status	R	workflow engine @srm/glb-workflow
workflow_state	R	workflow engine @srm/glb-workflow

Object	R/W	ใช้ของระบบเดิมตัวไหน
menus	R	auth-backend menus
email_template	R	email_template + email_sent + @gosoft-sbp/email-lib

9.7 Repository Providers + Module wiring

```
// src/providers/sgi/sgi.ts — repository provider แบบ factory (ไม่ใช้ TypeOrmModule.forFeature)
// convention ของโฟลเดอร์ providers คือ 1 ไฟล์ต่อโดเมน ตั้งชื่อตามโดเมน (business_user/business_user.ts,
// common_code/common_code.ts …) ไม่ใช่ index.ts
//
// ▲ ไฟล์นี้ใช้ร่วมกันทุกเอกสาร BE ของ SGI — ให้ **merge array เพิ่ม** เข้าไฟล์เดิม ห้ามเขียนทับ
// (ชื่อ const แยกต่อเอกสารไว้แล้วเพื่อไม่ให้ชนกัน)
import { DataSource } from 'typeorm';
import { ImpactedStore } from '../../entitys/sgi-impacted-stores.entity';
import { CommonCode } from '../../entitys/common-code.entity';
import { BusinessUser } from '../../entitys/business-user.entity';

export const sgiLookupProviders = [
 {
  provide: 'SGI_IMPACTED_STORE_REPOSITORY',
  useFactory: (dataSource: DataSource) => dataSource.getRepository(ImpactedStore),
  inject: ['DATA_SOURCE'],
 },
 {
  provide: 'COMMON_CODE_REPOSITORY',
  useFactory: (dataSource: DataSource) => dataSource.getRepository(CommonCode),
  inject: ['DATA_SOURCE'],
 },
 {
  provide: 'BUSINESS_USER_REPOSITORY',
  useFactory: (dataSource: DataSource) => dataSource.getRepository(BusinessUser),
  inject: ['DATA_SOURCE'],
 },
];

// src/modules/sgi-lookup/sgi-lookup.module.ts
import { MiddlewareConsumer, Module, NestModule } from '@nestjs/common';
import { DatabaseModule } from '../../database/database.module';
// UserContextMiddleware อ่าน header x-user-id แล้วเซ็ต request.userId ที่ @UserId() ใช้
// — app.module.ts **ไม่ได้** apply แบบ global (มีแค่ HttpContext/LoggerContext) แต่ละโมดูลต้อง apply เอง
// (ดู evaluation-process.module.ts / inform-evaluate.module.ts / cooperation-request.module.ts)
import { UserContextMiddleware } from '../../common/middleware/user-context.middleware';
import { WorkflowModule } from '../workflow/workflow.module';
import { sgiLookupProviders } from '../../providers/sgi/sgi';
import { SgiLookupController } from './sgi-lookup.controller';
import { SgiLookupService } from './sgi-lookup.service';

@Module({
 imports: [DatabaseModule, WorkflowModule],
 controllers: [SgiLookupController],
 providers: [SgiLookupService, ...sgiLookupProviders],
 exports: [SgiLookupService],
})
export class SgiLookupModule implements NestModule {
 configure(consumer: MiddlewareConsumer) {
  // ถ้าไม่ apply ตรงนี้ userId จะเป็น undefined เงียบ ๆ ทุก endpoint
  consumer.apply(UserContextMiddleware).forRoutes(SgiLookupController);
 }
}
// TODO: register module นี้ใน app.module.ts (imports) พร้อมกับโมดูล SGI ตัวอื่น
```

9.8 BFF Proxy (module + controller + client service)

BFF ยังไม่มีฟีเจอร์ประกันรายได้เลย จึงต้องสร้าง module ใหม่ + client service ใหม่ทั้งชุด และเลือก prefix แบบเดียวทั้งโมดูล (ที่นี่ใช้ /bff/sgi/…) เพื่อไม่ให้ปนแบบที่มี/ไม่มี `/bff` เหมือนโมดูลเดิม

```
// BFF — ไม่ต้องมี client ของ SGI แยก
//
// ● แก้ 2026-09-22 — skeleton เดิมเสนอ `SgiClientService` เป็น class แยกที่ *ชี้ไป store backend*
// ของจริงในรีโปกลับชี้ `config.api.sgi.url` (ค่าตั้งต้น http://localhost:3010)
// ซึ่ง **ไม่มี backend นั้นอยู่จริง** และ `API_SGI_BACKEND_URL` ไม่ได้ตั้งไว้ที่ไหนเลย
// → 12 เส้น (รวม 5 เส้นที่ FE เรียกจริง) ยิงไม่ถึงปลายทาง (พบตอนรันจริง)
// มติ DP-10 ระบุว่า SGI **ไม่แยก backend** — อยู่ใน srm-sps-spsap-store-backend
// จึงใช้ `StoreClientService` ที่มีอยู่แล้วตรง ๆ ไม่สราง client ตัวที่สอง
// (`tools/check_docs.py` มีกฎห้ามโมดูล SGI ของ BFF ใช้ SgiClientService)
//
// header: ใช้ `OperatingManagementService.header(req)` ชุดกลางเท่านั้น ห้ามประกอบเอง
```

```
// ตัวประกอบที่เขียนเองในโมดูลส่ง header ไม่ครบ (ขาด x-user-full-name / accept-language)
// และ serialize x-user-permissions คนละรูปแบบกับฝั่งรับที่ใช้จริง (JSON.parse)
//
// path ที่ forward ให้ใช้ `/api/v1/sgi/...` เสมอ — controller ฝั่ง BE บางตัวประกาศ base path
// เป็น array 2 ค่า แต่บางตัวมีเฉพาะ canonical
// BaseClientService และ { success, data } ให้แล้ว — service ฝั่ง BFF จึงได้ data ตรง ๆ
// src/modules/sgi-lookup/sgi-lookup.service.ts (BFF)
import { Injectable } from '@nestjs/common';
import { StoreClientService } from '@common/client-services/store-client.service';
import { OperatingManagementService } from '@modules/operating-management/operating-management.service';

@Injectable()
export class SgiLookupBffService {
  constructor(
    private readonly client: StoreClientService,
    private readonly operatingManagementService: OperatingManagementService,
  ) {}

  // BFF ไม่มี DB — หน้าที่เดียวคือแนบ user context แล้ว forward
  // ▲ ต้อง unwrap envelope ของ store-backend 1 ชั้นก่อนคืน (ยืนยันจากโค้ดจริง 2026-09-04):
  // ResponseInterceptor ระดับ global ของ BFF ห่อผลลัพธ์เป็น { success, data, requestId } อีกที
  // ถ้าคืน { success, data } ดิบมา FE จะได้ data.data.data — BaseClientService จึงต้องคืน .data.data
  //
  // ● แก้ 2026-09-22 — เติม skeleton ให้ประกอบ header เองด้วย `userHeaders(user)` 3 ตัว
  // ส่งไม่ครบตาม LLDD-BE-Integration-SBP-Platform.pdf §5.1 (ขาด x-user-full-name · accept-language)
  // และ serialize x-user-permissions เป็น comma-separated ขณะที่ฝั่งรับที่ใช้จริง
  // (OperatingManagementHeaderGuard.parsePermissions) ทำ JSON.parse — คนละรูปแบบกัน
  // ให้ใช้ตัวประกอบกลางตัวเดียวทั้งระบบ และรับ `req` ไม่ใช่ `req.user`
  async getSgiLookupDocumentStatuses(params: any, req: any) {
    return this.client.get('/api/v1/sgi/lookup/document-statuses', { params, headers: await this.operatingManagementService.header(req) });
  }

  async getSgiLookupWorkflowSections(params: any, req: any) {
    return this.client.get('/api/v1/sgi/lookup/workflow-sections', { params, headers: await this.operatingManagementService.header(req) });
  }
}

// ---------- src/modules/sgi-lookup/sgi-lookup.controller.ts (BFF) ----------
import { Body, Controller, Delete, Get, Param, Post, Put, Query, Req, UseGuards } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';

// path เดียวกับที่ FE เรียก (apiClient baseURL รวม /api/v1 แล้ว) — ห้ามตั้งตามชื่อเอกสาร LLDD
@Controller('sgi/lookup')
@UseGuards(AuthGuard('jwt'))
export class SgiLookupBffController {
  constructor(private readonly service: SgiLookupBffService) {}

  // proxy ของ GET /api/v1/sgi/lookup/document-statuses
  @Get('document-statuses')
  getSgiLookupDocumentStatuses(@Query() query: any, @Req() req: any) {
    return this.service.getSgiLookupDocumentStatuses(query, req);
  }

  // proxy ของ GET /api/v1/sgi/lookup/workflow-sections
  @Get('workflow-sections')
  getSgiLookupWorkflowSections(@Query() query: any, @Req() req: any) {
    return this.service.getSgiLookupWorkflowSections(query, req);
  }
}
// TODO: register module ใน app.module.ts ของ BFF · imports ต้องมี ClientServiceModule + OperatingManagementModule
```

10. Database SQL

10.1 ตารางที่อ่าน/เขียน

Table / Object	R/W	Usage
sgi_impacted_stores	R	store picker — SGI ไม่มีตาราง stores ของตัวเอง
common_code	R	● แก้ 2026-09-16 — lookup สถานะ 6 ค่าและขั้น 5 ค่าอ่านจาก common_code ไม่ใช่ sps_store.workflow_status เพราะตารางนั้นไม่มีคอลัมน์ลำดับ และ status_id เป็น integer surrogate ต่างกันตาม environment · วงเงินอนุมัติอยู่ที่ code_name ของ SGI_APPROVE_LIMIT

Table / Object	R/W	Usage
business_user	R	popup ค้นหาพนักงาน — SGI ไม่มีตาราง employees
permissions	R	RBAC/menu matrix — จัดการที่หน้า /setting/manage-user-rights เดิม · SGI อ่านผ่าน header x-user-permissions เท่านั้น ไม่มีตารางของตัวเอง
email_sent	W (โดย email-lib)	log การส่ง — SGI เรียก sendEmail() ด้วยเลข template จาก workflow_route.email_id แล้ว lib เขียนแถวให้เอง (DP-5 · 2026-08-14) · ▲ คอลัมน์ผู้ส่งคือ send_by ไม่ใช่ sent_by
workflow_status	R	ใช้ของระบบเดิม: workflow engine @srm/glb-workflow
workflow_state	R	ใช้ของระบบเดิม: workflow engine @srm/glb-workflow
menus	R	ใช้ของระบบเดิม: auth-backend menus
email_template	R	ใช้ของระบบเดิม: email_template + email_sent + @gosoft-sbp/email-lib

10.2 SQL จริงต่อ Endpoint

GET /api/v1/sgi/lookup/document-statuses — รายการสถานะเอกสาร verbatim

```
-- ● แก้ 2026-09-16 — ของเดิมรันไม่ได้: sps_store.workflow_status **ไม่มีคอลัมน์ seq**
-- (ตารางจริงมีแค่ status_id · status_name · create_date · version_id)
-- และ status_id เป็น integer surrogate ต่างกันตาม environment ใช้เป็น statusCode ไม่ได้
-- รหัส 2 ตัวอักษร (06/08/01/02/03/99) เป็นของ SGI อยู่ที่ common_code · seed แล้ว
SELECT code_value AS status_code, code_name AS status_name, seq_no AS sort_order
FROM sps_store.common_code
WHERE code_type = 'SGI_DOC_STATUS' AND active_flag = 'Y'
ORDER BY seq_no;
```

GET /api/v1/sgi/lookup/workflow-sections — รายการ section 5 ขั้น

```
-- ● แก้ 2026-09-16 — ของเดิมรันไม่ได้ 2 จุด:
-- (1) c.code_value = s.state_id เทียบ varchar กับ integer · seed ใส่ code_value = '100000' ไม่ใช่รหัสขั้น
-- (2) อ่านยอดจาก other_value ซึ่งไม่ได้ seed — contract อยู่ที่ code_name
-- วงเงินเป็นค่าเดียวทั้งระบบ (เกณฑ์เดียว 100,000 · มติ 2026-08-18) ไม่ใช่ค่าแยกรายขั้น
-- ▲ BE ต้อง fail-fast เมื่อ threshold หาย/ซ้ำ/แปลงเป็นตัวเลขไม่ได้ — SQL อย่างเดียวทำแทนไม่ได้
SELECT c.code_value AS section_code, c.code_name AS section_name, c.seq_no AS sort_order,
       (SELECT CAST(l.code_name AS NUMERIC)
          FROM sps_store.common_code l
         WHERE l.code_type = 'SGI_APPROVE_LIMIT'
           AND l.code_value = '100000' AND l.active_flag = 'Y') AS approve_limit_amount
FROM sps_store.common_code c
WHERE c.code_type = 'SGI_DOC_STATUS' AND c.active_flag = 'Y'
  AND c.code_value <> '99' -- 99 = เสร็จสิ้น ไม่ใช่ขั้นที่รอคนทำ
ORDER BY c.seq_no;
```

10.3 Index / Constraint ที่ควรมี (ข้อเสนอ)

ยังไม่มีข้อมูลเงื่อนไข query พอจะเสนอ index — รอ SQL ต่อ endpoint ครบก่อน

11. Processing Flow

Step	Description
1	Validate query
2	Read/write table by domain
3	Return standard envelope for list endpoints

12. Acceptance Criteria

- status label ต้องเป็น verbatim
- permission mutation ต้อง audit
- SGI เรียก email-lib ส่งอีเมลเอง โดยใช้เลข template จาก workflow_route.email_id (ปิด DP-5 · 2026-08-14)
- Auth Group 1 เป็น platform/external reference ไม่ใช่งาน implement ใน LLDD นี้

13. Developer Test Checklist

No	Test
1	store lookup
2	status lookup
3	permission save without reason
4	email template reset
5	(ตัดออก) audit log search

14. Unit Test Scope

3 ชั่วโมง (30% ของ implementation 10 ชั่วโมง) · เครื่องมือ: Jest + mock repository/DataSource (ไม่ต่อ DB จริง)

หัวข้อนี้คือ unit test ที่ต้องเขียนคู่กับโค้ด — ต่างจาก *Developer Test Checklist* ซึ่งเป็น scenario ระดับ end-to-end/manual ที่ใช้ตอนตรวจรับ · รายการด้านล่าง derive จาก field/validation, acceptance criteria, endpoint และตารางที่เอกสารนี้เขียน

สิ่งที่ทดสอบ	ประเภท	เกณฑ์ผ่าน
`type`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: required for /store/search (ระบบ SBP เดิม) · รูปแบบ: impacted|new
`roleCode`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: required for permission · รูปแบบ: 00-10
`menuCode`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: required for permission · รูปแบบ: string
`templateCode`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: required · รูปแบบ: EM-01..EM-08
`reason`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: ไม่บังคับแล้ว · รูปแบบ: text
business rule	logic	status label ต้องเป็น verbatim
business rule	logic	permission mutation ต้อง audit
business rule	logic	SGI เรียก email-lib ส่งอีเมลเอง โดยใช้เลข template จาก workflow_route.email_id (ปิด DP-5 · 2026-08-14)
business rule	logic	Auth Group 1 เป็น platform/external reference ไม่ใช่งาน implement ใน LLDD นี้
GET /store/search (ระบบ SBP เดิม)	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง
`GET /api/v1/sgi/lookup/document-statuses`	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง

สิ่งที่ทดสอบ	ประเภท	เกณฑ์ผ่าน
`GET /api/v1/sgi/lookup/workflow-sections`	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง
email_sent (SBP เดิม)	transaction	จำลอง error กลางทาง แล้วยืนยันว่า rollback ครบ ไม่เหลือแถวค้าง (mock DataSource/QueryRunner)
service	error mapping	แปลง error ของ repository/lib เป็น error code ตามสัญญากลาง (LLDD-BE-API-Common-Contracts.pdf)

- ทุกเคสต้องรันได้โดยไม่ต่อ DB/บริการภายนอกจริง — mock ที่ขอบ repository/client เสมอ
- ข้อความไทยที่ยืนยันในเทสต้องเป็น verbatim ตาม ข้อกำหนดทางธุรกิจ ห้ามพิมพ์ใหม่
- เกณฑ์ผ่านของ CI: ทุกเคสในตารางนี้มี test จริงและผ่านทั้งหมด